WIRELESS COMMUNICATION NETWORKS AND SYSTEMS

LAB 04

Error Detection & Coding

Learning Objectives

By the end of this lab, students will be able to:

- Explain the purpose of redundancy for error detection and (basic) correction.
- Implement and test: single parity bit, byte-wise checksum, and CRC (CRC-8 & CRC-16).
- Simulate a noisy binary channel and measure detection performance (detected vs. undetected errors).
- (Optional) Encode/decode & correct single-bit errors with Hamming(7,4).

Pedagogical Context (lightweight)

We stay practical: build redundancy bits from scratch, flip bits in a simulated channel, then verify frames at the receiver. Focus is on **detection capability** (what patterns of errors are caught or missed) rather than heavy math.

Tools

Python 3.x with `numpy` (only for RNG convenience) and `matplotlib` (optional for plots).

Starter Cell: Imports & Helpers

Paste once at the top of your notebook/script.

```python
import numpy as np
rng = np.random.default_rng(123)

# --- Bit/byte utilities ---
def bytes_to_bits(b: bytes):
    return [(byte >> i) & 1 for byte in b for i in range(7, -1, -1)]

def bits_to_bytes(bits):
    # pad to multiple of 8
    if len(bits) % 8 != 0:
        bits = bits + [0] * (8 - len(bits) % 8)
    out = bytearray()
    for i in range(0, len(bits), 8):
        byte = 0
```

Copyright © WCNS Team 2026/2027

```
        for j, bit in enumerate(bits[i:i+8]):
            byte |= (bit & 1) << (7 - j)
        out.append(byte)
    return bytes(out)

def flip_bit(x: int, i: int) -> int:
    """Flip bit i (0=LSB) of integer x."""
    return x ^ (1 << i)

# --- Channel models ---
def bsc(bits, p, rng=rng):
    """Binary Symmetric Channel: flips each bit with prob p."""
    flips = rng.random(len(bits)) < p
    return [(b ^ int(f)) for b, f in zip(bits, flips)]

def burst_channel(bits, burst_len=8, burst_prob=0.05, rng=rng):
    """Simple burst: with prob, pick start and flip burst_len consecutive bits."""
    out = bits[:]
    if rng.random() < burst_prob and len(bits) >= burst_len:
        start = rng.integers(0, len(bits) - burst_len + 1)
        for i in range(start, start + burst_len):
            out[i] ^= 1
    return out

# --- Framing ---
def make_payload(n_bytes=32, mode="random", rng=rng):
    if mode == "random":
        return bytes(rng.integers(0, 256, size=n_bytes, dtype=np.uint8))
    if mode == "text":
        s = "Hello wireless students! "  # will be truncated/padded
        data = (s * ((n_bytes//len(s))+1)).encode()[:n_bytes]
        return data
    raise ValueError("unknown mode")
```

Step 1 — Parity Bit (Even/Odd)

Idea. Append 1 bit so total number of 1s is even (even parity) or odd (odd parity). Detects any odd number of bit flips in protected block.

1A. Byte-wise Even Parity

```
def even_parity_bit(byte: int) -> int:
    """Return parity bit (0/1) such that total ones (byte+parity) is even."""
    ones = bin(byte).count("1")
    return ones & 1  # 1 if odd, 0 if even

def add_parity_frame(payload: bytes, parity="even"):
    bits = []
    for b in payload:
        # 8 data bits
        for i in range(7, -1, -1):
            bits.append((b >> i) & 1)
        # parity bit
        p = even_parity_bit(b)
        if parity == "even":
            bits.append(p)  # ensures even total ones
        else:
```

Copyright © WCNS Team 2026/2027

```
            bits.append(p ^ 1)  # odd parity if requested
    return bits

def check_parity_frame(bits, parity="even"):
    assert len(bits) % 9 == 0
    ok = True
    for i in range(0, len(bits), 9):
        data = bits[i:i+8]
        pbit = bits[i+8]
        ones = sum(data) + pbit
        if parity == "even":
            ok &= (ones % 2 == 0)
        else:
            ok &= (ones % 2 == 1)
    return ok
```

1B. Quick Test

```
pl = make_payload(16, "text")
frame = add_parity_frame(pl, parity="even")
# Send through BSC with p=0.02
rx = bsc(frame, p=0.02)
print("Parity check passed:", check_parity_frame(rx, "even"))
```

Note.. Two bit flips in the same protected block may go undetected (even number of errors).

Step 2 — Simple Checksums

Idea. Sum all bytes (with wrap-around), then append the sum (or its ones' complement). Catches many errors but not all.

2A. 8-bit Modular Sum (Wrap to 8 bits)

```
def checksum8(payload: bytes) -> int:
    s = sum(payload) & 0xFF
    return s

def add_checksum8_frame(payload: bytes):
    cs = checksum8(payload)
    return list(bytes(payload + bytes([cs]))), cs  # return byte-level frame

def verify_checksum8_frame(frame_bytes: bytes) -> bool:
    data, cs = frame_bytes[:-1], frame_bytes[-1]
    return checksum8(data) == cs
```

2B. Ones' Complement Checksum (Internet-style)

```
def ones_complement_add(a, b, width=16):
    mask = (1 << width) - 1
    s = a + b
    # wrap-around carry
    return (s & mask) + (s >> width)
```

Copyright © WCNS Team 2026/2027

```python

def checksum16_ones_complement(payload: bytes) -> int:
    # pad to 16-bit words
    if len(payload) % 2 == 1:
        payload += bytes([0])
    s = 0
    for i in range(0, len(payload), 2):
        word = (payload[i] << 8) | payload[i+1]
        s = ones_complement_add(s, word, width=16)
    return (~s) & 0xFFFF  # ones' complement

def add_checksum16_frame(payload: bytes):
    cs = checksum16_ones_complement(payload)
    frame = payload + bytes([cs >> 8, cs & 0xFF])
    return list(frame), cs

def verify_checksum16_frame(frame_bytes: bytes) -> bool:
    data, csH, csL = frame_bytes[:-2], frame_bytes[-2], frame_bytes[-1]
    cs = (csH << 8) | csL
    return checksum16_ones_complement(data) == cs
```

Undetected example.. Some paired changes can preserve the sum; checksums are not as strong as CRCs.

Step 3 — CRC (Cyclic Redundancy Check)

Idea. Polynomial division over GF(2). Strong burst-error detection; widely used (Ethernet, LTE/5G channels use variants).

3A. Generic Bitwise CRC Engine

```python
def crc_bits(data_bits, poly, init=0, width=8, refin=False, refout=False, xorout=0):
    """
    Simple bitwise CRC on bits (MSB-first if refin=False).
    poly: generator polynomial without the top bit (e.g., 0x07 for CRC-8 x^8+x^2+x+1)

    width: CRC width in bits
    """
    reg = init
    topbit = 1 << (width - 1)
    for b in data_bits:
        bit = b & 1
        if refin:
            # LSB-first input
            carry = (reg & 1) ^ bit
            reg >>= 1
            if carry:
                reg ^= (poly << (width-8))  # assumes poly fits 8 bits for width>=8
        else:
            # MSB-first input
            carry = ((reg & topbit) != 0) ^ (bit != 0)
            reg = ((reg << 1) & ((1<<width)-1))
            if carry:
                reg ^= poly
    if refout:
        # reflect output if required (not used here)
```

```
        out = 0
        for i in range(width):
            out |= ((reg >> i) & 1) << (width-1-i)
        reg = out
    return reg ^ xorout

# Convenience wrappers for two common CRCs (simple params)
def crc8(data_bits):
    # CRC-8 poly x^8 + x^2 + x + 1 -> 0x07 (MSB-first)
    return crc_bits(data_bits, poly=0x07, init=0x00, width=8)

def crc16_ccitt(data_bits):
    # CRC-16-CCITT (X^16 + X^12 + X^5 + 1): 0x1021 (MSB-first), init 0xFFFF common
    return crc_bits(data_bits, poly=0x1021, init=0xFFFF, width=16)
```

3B. Framing with CRC

```
def add_crc8_frame(payload: bytes):
    data_bits = bytes_to_bits(payload)
    c = crc8(data_bits)
    crc_bits8 = [(c >> i) & 1 for i in range(7, -1, -1)]
    frame_bits = data_bits + crc_bits8
    return frame_bits, c

def verify_crc8_frame(frame_bits):
    data = frame_bits[:-8]
    recv_crc_bits = frame_bits[-8:]
    recv_crc = 0
    for i, b in enumerate(recv_crc_bits):
        recv_crc |= (b & 1) << (7 - i)
    return crc8(data) == recv_crc

def add_crc16_frame(payload: bytes):
    data_bits = bytes_to_bits(payload)
    c = crc16_ccitt(data_bits)
    crc_bits16 = [(c >> i) & 1 for i in range(15, -1, -1)]
    frame_bits = data_bits + crc_bits16
    return frame_bits, c

def verify_crc16_frame(frame_bits):
    data = frame_bits[:-16]
    recv_crc_bits = frame_bits[-16:]
    recv_crc = 0
    for i, b in enumerate(recv_crc_bits):
        recv_crc |= (b & 1) << (15 - i)
    return crc16_ccitt(data) == recv_crc
```

Facts (to discuss).. CRC-16 detects all single-bit, double-bit errors, any odd number of bit errors (for some polynomials), and all bursts shorter than its degree (here up to 16 bits).

Step 4 — Monte Carlo: Detection Performance

Run many frames through noisy channel; measure detection rates for each scheme.

Copyright © WCNS Team 2026/2027

4A. Experiment Harness

```python
def run_trial(payload_len=64, p=0.01, scheme="parity", rng=rng):
    payload = make_payload(payload_len, "random", rng)
    if scheme == "parity":
        tx_bits = add_parity_frame(payload, "even")
        rx_bits = bsc(tx_bits, p, rng)
        ok = check_parity_frame(rx_bits, "even")
        # true error?
        err = (tx_bits != rx_bits)
        return ok, any(err)
    elif scheme == "cs8":
        tx = bytes(payload + bytes([checksum8(payload)]))
        # send as bytes -> bits for BSC
        tx_bits = bytes_to_bits(tx)
        rx_bits = bsc(tx_bits, p, rng)
        rx = bits_to_bytes(rx_bits)
        ok = verify_checksum8_frame(rx)
        err = (tx != rx)
        return ok, (tx != rx)
    elif scheme == "cs16":
        frame, _ = add_checksum16_frame(payload)
        tx = bytes(frame)
        tx_bits = bytes_to_bits(tx)
        rx_bits = bsc(tx_bits, p, rng)
        rx = bits_to_bytes(rx_bits)
        ok = verify_checksum16_frame(rx)
        return ok, (tx != rx)
    elif scheme == "crc8":
        tx_bits, _ = add_crc8_frame(payload)
        rx_bits = bsc(tx_bits, p, rng)
        ok = verify_crc8_frame(rx_bits)
        return ok, (tx_bits != rx_bits)
    elif scheme == "crc16":
        tx_bits, _ = add_crc16_frame(payload)
        rx_bits = bsc(tx_bits, p, rng)
        ok = verify_crc16_frame(rx_bits)
        return ok, (tx_bits != rx_bits)
    else:
        raise ValueError("unknown scheme")

def evaluate(Ntrials=200, payload_len=64, p=0.01):
    schemes = ["parity","cs8","cs16","crc8","crc16"]
    results = {}
    for sch in schemes:
        detected = undetected = clean = 0
        for _ in range(Ntrials):
            ok_flag, any_error = run_trial(payload_len, p, sch, rng)
            if not any_error:
                clean += 1
            else:
                # any_error==True means channel introduced at least 1 bit error
                if ok_flag:
                    # scheme says "OK" despite error -> undetected!
                    undetected += 1
                else:
                    detected += 1
        results[sch] = {
            "trials": Ntrials,
            "clean_frames": clean,
```

Copyright © WCNS Team 2026/2027

```
            "errored_frames": Ntrials - clean,
            "detected_errors": detected,
            "undetected_errors": undetected,
            "undetected_rate": undetected / max(1, (Ntrials - clean))
        }
    return results

res = evaluate(Ntrials=300, payload_len=128, p=0.01)
for k,v in res.items():
    print(k, v)
```

4B. Suggested Variations

- Change p (e.g., 0.001, 0.01, 0.05) and compare `undetected_rate`.
- Replace BSC by `burst_channel` and discuss which scheme better handles bursts (CRC wins).

Step 5 — (Optional) Hamming(7,4) Single-Bit Correction

Idea. Encode 4 data bits into 7 with parity checks that allow correction of any single-bit error in a codeword.

5A. Encoder/Decoder

```
# Positions: [1..7] with parity bits at 1,2,4 (classic convention)
P = [1,2,4]
D = [3,5,6,7]

def hamming74_encode(nibble4):  # nibble4: list of 4 bits [d1,d2,d3,d4]
    code = [0]*7
    code[2], code[4], code[5], code[6] = nibble4  # positions 3,5,6,7
    # parity bits (even)
    code[0] = (code[2] ^ code[4] ^ code[6]) & 1  # p1 covers 1,3,5,7
    code[1] = (code[2] ^ code[5] ^ code[6]) & 1  # p2 covers 2,3,6,7
    code[3] = (code[4] ^ code[5] ^ code[6]) & 1  # p4 covers 4,5,6,7
    return code

def hamming74_syndrome(code7):
    s1 = (code7[0] ^ code7[2] ^ code7[4] ^ code7[6]) & 1
    s2 = (code7[1] ^ code7[2] ^ code7[5] ^ code7[6]) & 1
    s4 = (code7[3] ^ code7[4] ^ code7[5] ^ code7[6]) & 1
    return s4<<2 | s2<<1 | s1

def hamming74_decode(code7):
    syn = hamming74_syndrome(code7)
    corr = code7[:]
    if syn != 0 and 1 <= syn <= 7:
        corr[syn-1] ^= 1  # correct single-bit error
    data = [corr[2], corr[4], corr[5], corr[6]]
    return data, syn
```

5B. Quick Demo

```
d = [1,0,1,1]
```

Copyright © WCNS Team 2026/2027

```
c = hamming74_encode(d)
# flip one random bit
pos = rng.integers(0,7)
c_err = c[:]; c_err[pos] ^= 1
d_hat, syn = hamming74_decode(c_err)
print("orig:", d, "flip_pos:", pos+1, "syndrome:", syn, "decoded:", d_hat)
```

Note.. Hamming(7,4) corrects any single-bit error but only detects (not correct) most double-bit errors.

Guiding Questions

1. Give an example where parity fails but CRC-16 succeeds. Why?
2. Why does a ones' complement checksum sometimes miss certain multi-bit patterns?
3. How does frame length affect undetected error probability for each scheme?
4. In practice, why do protocols combine CRC with FEC (e.g., convolutional/LDPC codes)?

Copyright © WCNS Team 2026/2027